Real-time Speech-to-Text Service

Hindi · English · Hinglish transcription with a CRUD dashboard

Technical Documentation

Kasinadh Sarma

October 2026

Live application	https://kasinadhsarma-speech-to-text.hf.space
Source code	https://github.com/kasinadhsarma/procucev-speech-to-text
Stack	Python 3.12, FastAPI, WebSockets, SQLAlchemy, PostgreSQL, faster-whisper

Contents

1 Overview **2**

2 System Architecture **2**
- 2.1 Component diagram . . . 2
- 2.2 Real-time sequence . . . 3
- 2.3 Segmentation algorithm . . . 3

3 Database Design **4**
- 3.1 Entity . . . 4
- 3.2 DDL . . . 4
- 3.3 Queries . . . 4
- 3.4 Connection handling . . . 5

4 API Specification **5**
- 4.1 REST endpoints . . . 5
- 4.2 Transcription schema . . . 5
- 4.3 WebSocket protocol . . . 5

5 Technology Stack **5**

6 Deployment **6**
- 6.1 Hugging Face Spaces . . . 6
- 6.2 Local and container options . . . 6
- 6.3 Configuration . . . 7

7 Testing **7**

8 Limitations and Future Work **7**

1 Overview

The service transcribes speech from the browser microphone *as the user speaks*. Speech may mix Hindi and English within one sentence. Each session is stored in a relational database, and a dashboard provides create, read, update, delete and search over the transcription history.

Requirements met

- **Real-time transcription:** partial text roughly every 1.5 s, finalised at pauses.
- **Multilingual:** Whisper with automatic language detection and a Hinglish decoder prompt.
- **Self-hosted open-source STT:** OpenAI Whisper weights run through faster-whisper; no third-party speech API.
- **Relational database:** PostgreSQL (Neon) in production, SQLite fallback for local runs.
- **Dashboard with CRUD:** history list, search, create, edit and delete.
- **Deployed on the cloud:** Hugging Face Spaces, served over HTTPS/WSS.

2 System Architecture

2.1 Component diagram

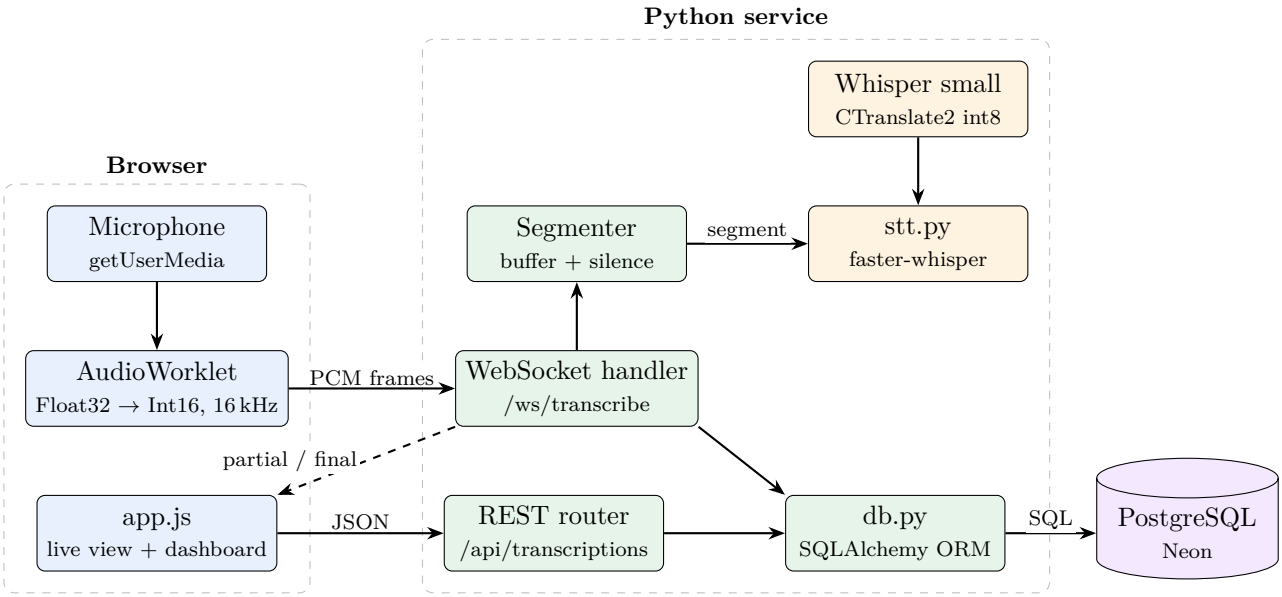


Figure 1: Main components and data flow.

Module	Responsibility
`static/app.js`	Microphone capture, WebSocket streaming, live transcript rendering, dashboard CRUD
`app/main.py`	FastAPI application: REST endpoints, WebSocket endpoint, segmentation loop, static hosting
`app/stt.py`	Lazily loads one shared Whisper model; `transcribe(audio, language)`
`app/db.py`	Engine and session setup, `Transcription` ORM model, `DATABASE_URL` normalisation
`hf_entry.py`	Hugging Face entry point: Gradio owns port 7860; FastAPI routes are injected ahead of Gradio's

Table 1: Source modules.

2.2 Real-time sequence

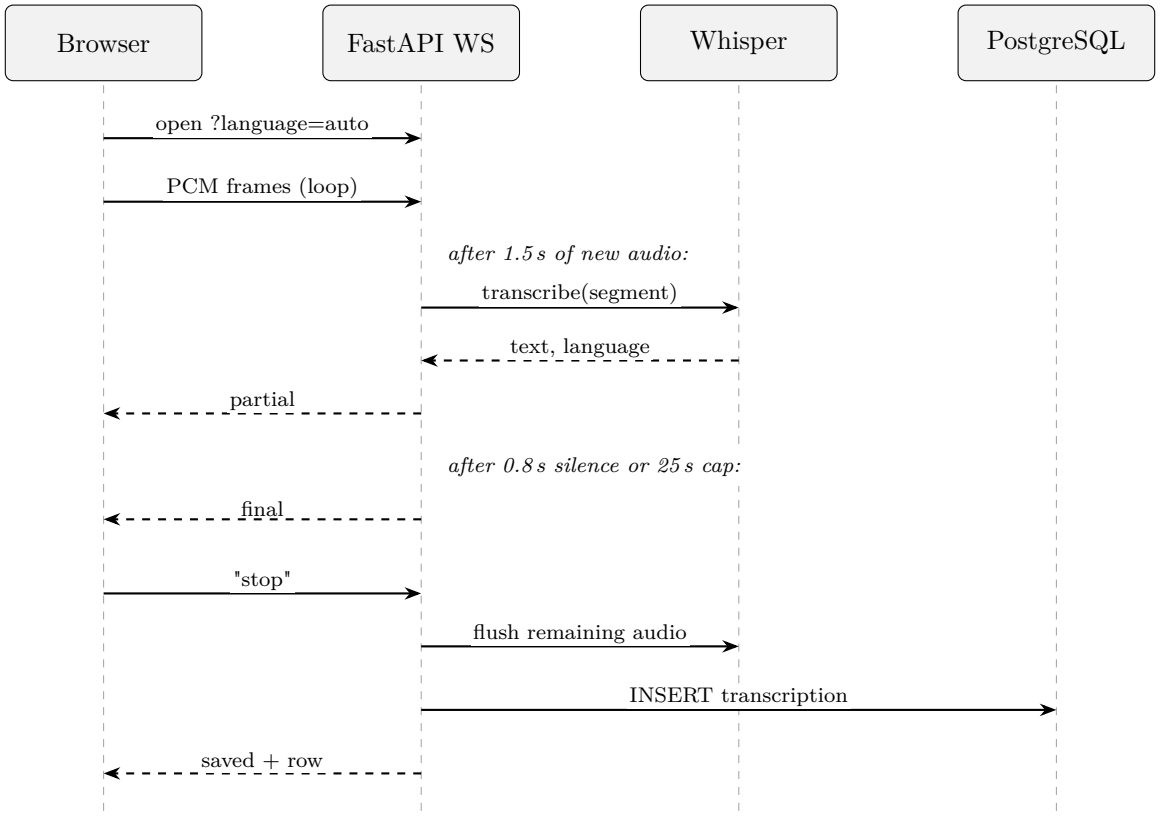


Figure 2: Streaming transcription over one WebSocket session.

2.3 Segmentation algorithm

Whisper is not a streaming model, so the server keeps a growing *segment* buffer and re-decodes it:

1. Append each incoming Int16 frame (scaled to Float32 in $[-1, 1]$) to the segment buffer.
2. If the last 0.8 s has RMS energy < 0.012 and the segment is longer than 1 s, or the segment exceeds 25 s: decode it, send a `final` message and clear the buffer.
3. Otherwise, if 1.5 s of new audio arrived since the last decode: decode and send a `partial` message.
4. On `stop`: decode what remains, join all finals, store one row and send `saved`.

Decoding runs in a worker thread (`asyncio.to_thread`) so the event loop keeps receiving audio.

Parameter	Value	Constant
Sample format	16 000 Hz, mono, Int16 LE	`SAMPLE_RATE`
Partial interval	1.5 s	`PARTIAL_EVERY_SEC`
Silence window / threshold	0.8 s / RMS 0.012	`SILENCE_SEC`, `SILENCE_RMS`
Maximum segment	25 s	`MAX_SEGMENT_SEC`
Decoder	beam 1, VAD filter, Hinglish prompt	`stt.transcribe`

Table 2: Real-time tuning parameters.

3 Database Design

3.1 Entity

transcriptions		
id	SERIAL	PK
title	VARCHAR(200)	default 'Untitled'
text	TEXT	default ''
language	VARCHAR(32)	default 'auto'
duration_sec	DOUBLE PRECISION	default 0
created_at	TIMESTAMPTZ	UTC
updated_at	TIMESTAMPTZ	UTC, on update

Figure 3: Table `transcriptions`.

3.2 DDL

```
CREATE TABLE transcriptions (
    id            SERIAL PRIMARY KEY,
    title         VARCHAR(200)      NOT NULL DEFAULT 'Untitled',
    text          TEXT              NOT NULL DEFAULT '',
    language      VARCHAR(32)       NOT NULL DEFAULT 'auto',
    duration_sec  DOUBLE PRECISION  NOT NULL DEFAULT 0,
    created_at    TIMESTAMPTZ       NOT NULL DEFAULT now(),
    updated_at    TIMESTAMPTZ       NOT NULL DEFAULT now()
);
```

Defaults are applied by the SQLAlchemy model; the table is created on startup by `Base.metadata.create_all()`.

3.3 Queries

Operation	SQL (simplified)
List	`SELECT * FROM transcriptions ORDER BY created_at DESC, id DESC LIMIT :limit OFFSET :offset`
Search	`... WHERE text ILIKE :p OR title ILIKE :p`
Read	`SELECT * FROM transcriptions WHERE id = :id`
Create	`INSERT INTO transcriptions (...) VALUES (...) RETURNING id`
Update	`UPDATE transcriptions SET ..., updated_at = :now WHERE id = :id`
Delete	`DELETE FROM transcriptions WHERE id = :id`

3.4 Connection handling

`DATABASE_URL` selects the backend (unset means SQLite). Before connecting, `app/db.py` strips whitespace and quotes, rewrites `postgres://` and `postgresql://` to `postgresql+psycopg2://`, removes Neon's `channel_binding=require` (TLS is still enforced by `sslmode=require`), and enables `pool_pre_ping`. `GET /api/health` reports the active backend.

4 API Specification

4.1 REST endpoints

Method	Path	Description	Success
GET	`/api/health`	Liveness, model, database backend	200
GET	`/api/transcriptions`	List; `q`, `limit` (1–200), `offset`	200
POST	`/api/transcriptions`	Create	201
GET	`/api/transcriptions/{id}`	Read one	200 / 404
PUT	`/api/transcriptions/{id}`	Partial update	200 / 404
DELETE	`/api/transcriptions/{id}`	Delete	204 / 404

4.2 Transcription schema

```
{
  "id": 1,
  "title": "Kal␣subah␣meeting␣hai...",
  "text": "Kal␣subah␣meeting␣hai,␣please␣slides␣ready␣rakhna.",
  "language": "hi",
  "duration_sec": 4.12,
  "created_at": "2026-10-22T11:40:02.118Z",
  "updated_at": "2026-10-22T11:40:02.118Z"
}
```

Create body: `title` (max 200, default `"Untitled"`), `text`, `language` (default `"auto"`), `duration_sec`. Update body: any subset of `title`, `text`, `language`. Errors return `{"detail": "..."}`; validation errors return 422.

4.3 WebSocket protocol

Endpoint: `wss://<host>/ws/transcribe?language=auto|hi|en`.

Direction	Frame	Meaning
client → server	binary	Int16 LE PCM, mono, 16 kHz, any chunk size
client → server	text `stop`	Flush, save and close
server → client	`{"type":"partial","text","language"}`	Provisional text, may change
server → client	`{"type":"final","text","language"}`	Stable text for a closed segment
server → client	`{"type":"saved","transcription"}`	Stored row, or `null` if nothing recognised
server → client	`{"type":"error","message"}`	Unexpected server error

5 Technology Stack

Library	Version	License	Purpose
FastAPI	0.159	MIT	REST and WebSocket endpoints, OpenAPI docs
Starlette	1.9	BSD-3-Clause	ASGI toolkit, static files, WebSockets
Uvicorn	0.54	BSD-3-Clause	ASGI server
Pydantic	2.14	MIT	Request validation
SQLAlchemy	2.1	MIT	ORM and database engine
psycopg2-binary	2.9	LGPL-3.0	PostgreSQL driver
faster-whisper	1.2	MIT	Whisper inference with VAD filter
CTranslate2	4.8	MIT	Optimised int8 CPU inference
NumPy	2.x	BSD-3-Clause	PCM buffers and RMS silence detection
Whisper *small*	–	MIT	244 M-parameter multilingual ASR model
Gradio	6.x	Apache-2.0	HTTP host on Hugging Face ZeroGPU
pytest, HTTPX	–	MIT, BSD-3	Tests
websockets	17.x	BSD-3-Clause	Streaming test client
PostgreSQL (Neon)	16	PostgreSQL	Production database

Frontend: HTML5, CSS3 and vanilla JavaScript using `getUserMedia`, `AudioContext`, `AudioWorklet`, `WebSocket`, `fetch` and `<dialog>`. There is no build step.

6 Deployment

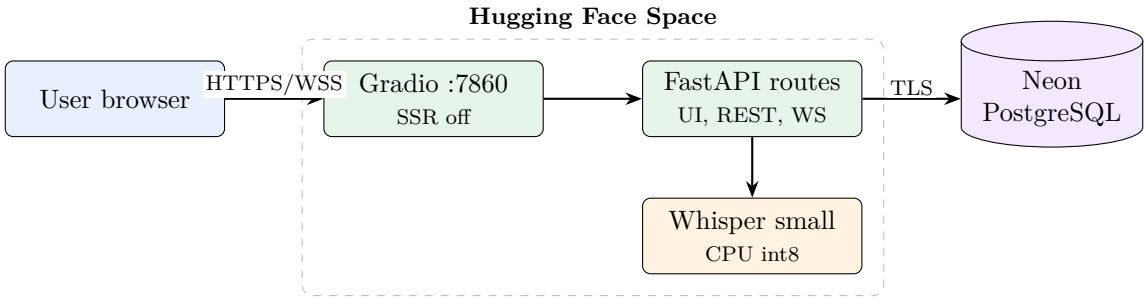


Figure 4: Production deployment.

6.1 Hugging Face Spaces

Free accounts get ZeroGPU hardware for Gradio-SDK Spaces, which only starts if Gradio launches an app containing an `@spaces.GPU` function. `hf_entry.py` therefore lets Gradio own port 7860 with a hidden placeholder function and passes the FastAPI routes through `app_kwargs`, so they take precedence. Server-side rendering is disabled because its Node proxy would hide those routes. Transcription runs on CPU. The Neon connection string is stored as the Space secret `DATABASE_URL`.

6.2 Local and container options

```
# Full stack with PostgreSQL
docker compose up --build              # http://localhost:8000

# Without Docker (SQLite)
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
.venv/bin/uvicorn app.main:app

# Tests
.venv/bin/pip install -r requirements-dev.txt && .venv/bin/pytest -q
```

6.3 Configuration

Variable	Default	Description
`DATABASE_URL`	SQLite file	SQLAlchemy URL
`WHISPER_MODEL`	`small`	`tiny`, `base`, `small`, `medium`, `large-v3`
`WHISPER_DEVICE`	`cpu`	`cpu` or `cuda`
`WHISPER_COMPUTE`	`int8`	CTranslate2 compute type
`WHISPER_THREADS`	4	Inference CPU threads
`WHISPER_PROMPT`	Hinglish sentence	Decoder initial prompt
`PRELOAD_MODEL`	1	Load model at startup
`PORT`	7860	HTTP port

7 Testing

- **Unit/API:** `tests/test_api.py` exercises create, read, search, update, delete and 404 handling against a temporary SQLite database, with Hindi (Devanagari) text.
- **End-to-end streaming:** `tests/stream_demo.py` streams a 16 kHz PCM recording of a mixed Hindi/English sentence in real time, locally or against the live `wss://` endpoint, and prints the partial, final and saved messages.

8 Limitations and Future Work

- No authentication; all visitors share one history. Next: user accounts and a `user_id` column.
- No migration tool; add Alembic.
- One shared model instance; concurrent sessions queue for CPU. Next: GPU inference or a worker pool.
- Accented English may be transliterated into Devanagari; larger models reduce this.
- Energy-based segmentation can mis-split in noisy rooms; a streaming VAD (e.g. Silero on the client) would help.
- Full-text search at scale: `pg_trgm` GIN index.